DAA PRACTICAL 8

Job Sequencing with Deadlines Using Greedy Method

Exactly 60 Viva Questions & Answers | Revision Guide

Core strategy: Sort jobs by descending profit. For each job, search backward from its deadline and place it in the latest empty slot. Skip it if no valid slot is available.

Worked example: A(2,100), B(1,19), C(2,27), D(1,25), E(3,15) gives schedule C A E and total profit 142.

Assumptions

Each job takes exactly one unit of time; one job can occupy a slot; a selected job must finish by its deadline; and each job is scheduled at most once. The classic greedy optimality claim relies on these assumptions.

A. Problem and greedy strategy

Q1. What is Job Sequencing with Deadlines?

Answer: It is an optimization problem in which each job has a deadline and a profit. We select and arrange jobs in time slots to maximize total profit while completing every selected job by its deadline.

Q2. What is the objective of this problem?

Answer: Maximize the total profit earned from the jobs scheduled without violating their deadlines.

Q3. What are the standard assumptions?

Answer: Each job takes exactly one unit of time, only one job can run in a slot, a job earns its profit only if completed by its deadline, and each job can be selected at most once.

Q4. Which algorithm is used in this practical?

Answer: A greedy algorithm. It considers jobs in descending order of profit and assigns each job to the latest available slot on or before its deadline.

Q5. What is a greedy algorithm?

Answer: It builds a solution through locally preferred choices according to a rule. For this problem, the rule is to consider higher-profit jobs first and schedule each in a feasible slot.

Q6. Why do we sort jobs by descending profit?

Answer: The objective is to maximize profit, so considering higher-profit jobs first gives priority to more valuable jobs. The slot-placement rule then checks whether each choice is feasible.

Q7. Why do we place a job in the latest available slot before its deadline?

Answer: Placing it as late as possible preserves earlier slots for jobs with tighter deadlines, increasing the chance that more jobs can be scheduled.

Q8. Why is the latest-slot greedy strategy optimal under the standard assumptions?

Answer: For unit-time jobs with deadlines and profits, an optimal schedule can be rearranged so selected jobs are in nondecreasing deadline order. Considering jobs by descending profit and placing each in the latest feasible free slot preserves feasibility and yields a maximum-profit schedule.

Q9. Why is `struct Job` used?

Answer: It groups the ID, deadline and profit of one job into a single record, so the information stays together when jobs are sorted.

Q10. Could we use a class instead of a struct?

Answer: Yes. Both can group data and functions. Struct members are public by default, while class members are private by default. Struct is convenient for this simple record because the code accesses its fields directly.

Q11. Why is the ID stored as `char`?

Answer: The sample uses single-character labels such as A, B and C. If IDs could contain longer names or strings, `string` would be more suitable.

Q12. What is the purpose of `compare()`?

Answer: It compares two Job records and returns true when the first job has a higher profit than the second, allowing sort to arrange jobs in descending profit order.

B. Data structures, sorting and code

Q13. Why does `compare()` return `a.profit > b.profit`?

Answer: The greater-than comparison places the job with the larger profit earlier in the sorted sequence.

Q14. What does `sort(jobs, jobs + n, compare)` do?

Answer: It sorts the half-open range from jobs through jobs+n, using compare to order the records. The entire Job record moves together, so each ID remains associated with its deadline and profit.

Q15. Which header provides `sort()`, `max()` and `min()`?

Answer: The `<algorithm>` header provides these standard algorithms and function objects.

Q16. What does `Job jobs[n]` represent?

Answer: It declares an array intended to hold n Job records. A variable-length array is not standard C++; some compilers accept it as an extension. `vector<Job> jobs(n)` is the portable alternative.

Q17. Why do we input all jobs before sorting?

Answer: The program needs each job's profit to determine the order and its deadline to decide where it can be placed. Therefore, it reads all records before sorting and scheduling.

Q18. What is `maxDeadline`?

Answer: It stores the largest deadline among all input jobs. It determines the number of time slots the program needs to represent.

Q19. Why do we find the maximum deadline?

Answer: No job can be scheduled after its own deadline, so the maximum deadline is the latest slot that could ever be useful. Allocating slots only up to this value avoids unnecessary slots.

Q20. What does `max(maxDeadline, jobs[i].deadline)` do?

Answer: It compares the current maximum deadline with the current job's deadline and keeps the larger value.

Q21. Why is the slot array sized `maxDeadline + 1`?

Answer: The code uses one-based slot numbering, from 1 through maxDeadline. Index 0 is deliberately unused, so the array needs one extra element.

Q22. What does `'-'` mean in the slot array?

Answer: It is a marker meaning that a time slot is currently empty. A job ID replaces the marker when that slot is occupied.

Q23. Why is `totalProfit` initialized to zero?

Answer: Initially, no job has been scheduled, so the accumulated profit is zero. The program adds a job's profit only after successfully assigning it a slot.

Q24. What does `jobCount` store?

Answer: It counts the number of jobs successfully scheduled. It increases only when a free slot is found and filled.

Q25. Explain the outer scheduling loop.

Answer: The outer loop visits each job once in descending profit order. For each job, the inner loop searches for a valid empty slot.

Q26. Why does the inner loop start at `min(maxDeadline, jobs[i].deadline)`?

Answer: A job cannot be placed after its deadline. The minimum also ensures the starting slot does not exceed the allocated maximum deadline.

Q27. Why does the inner loop decrement j?

Answer: It searches backward from the latest permissible slot toward slot 1, so the job is placed as late as possible.

C. Scheduling logic and dry run

Q28. What does `slot[j] == '-'` check?

Answer: It checks whether the current time slot is unoccupied and therefore available for the job.

Q29. What does `slot[j] = jobs[i].id` do?

Answer: It assigns the current job's ID to the selected time slot, marking that slot as occupied.

Q30. Why do we add profit only after finding an empty slot?

Answer: A job earns profit only if it is actually scheduled within its deadline. An unscheduled job must not contribute to totalProfit.

Q31. Why is `break` used after scheduling a job?

Answer: Once a job is assigned to one slot, the inner search must stop. Otherwise, the same job could be placed in multiple slots.

Q32. What happens if no slot is available before a job's deadline?

Answer: The inner loop finishes without scheduling that job. The program skips it and moves to the next lower-profit job.

Q33. Why does the display loop start at slot 1?

Answer: Index 0 is unused. The valid time slots are numbered from 1 through maxDeadline.

Q34. Why might the displayed job order differ from the sorted profit order?

Answer: The program prints jobs by chronological slot order, not by the order in which they were considered. A later-considered job may occupy an earlier time slot.

Q35. Work through the example: A(2,100), B(1,19), C(2,27), D(1,25), E(3,15).

Answer: After sorting: A, C, D, B, E. There are three slots because the maximum deadline is 3. A goes to slot 2; C goes to slot 1; D and B cannot fit; E goes to slot 3.

Q36. What is the final schedule for that example?

Answer: Slot 1 = C, slot 2 = A, slot 3 = E. The displayed scheduled jobs are C A E.

Q37. What is the total profit in the example?

Answer: 100 + 27 + 15 = 142. Three jobs are scheduled.

Q38. What is the time complexity of sorting?

Answer: O(n log n) comparisons for sorting n jobs using the standard C++ sort algorithm.

Q39. What is the time complexity of finding the maximum deadline?

Answer: O(n), because the program checks each job's deadline once.

D. Complexity and optimization

Q40. What is the worst-case time complexity of the scheduling loops?

Answer: O(nD), where D is the maximum deadline. In the worst case, each of n jobs may scan up to D slots.

Q41. What is the overall time complexity?

Answer: O(n log n + nD), where n is the number of jobs and D is the maximum deadline. The scheduling scan may dominate when D is large.

Q42. What is the space complexity?

Answer: O(n + D): O(n) for storing the jobs and O(D) for the slot array. The sorting implementation may use additional implementation-dependent auxiliary space.

Q43. What is the limitation of the basic slot-scanning approach?

Answer: For every job, it may scan many slots backward. If n and D are large, this O(nD) scheduling work can be expensive.

Q44. How can Disjoint Set Union improve slot searching?

Answer: A DSU can represent the latest free slot at or before a given deadline. After occupying a slot, union it with the preceding slot. With path compression and union by rank/size, slot lookup is nearly constant amortized time, typically $O(\alpha(D))$ per operation.

Q45. What is the role of the DSU parent value in this optimization?

Answer: The representative of a slot indicates the latest available slot at or before that position. If the representative is 0, no valid free slot remains.

Q46. Can we sort jobs by deadline instead of profit?

Answer: Sorting by deadline alone does not prioritize the objective of maximizing profit. The standard greedy strategy prioritizes profit, then uses deadlines to find feasible slots.

Q47. What if two jobs have the same profit?

Answer: Either can be considered first for the basic method. A tie-breaking rule, such as earlier deadline first, may be used for deterministic behavior, but the core profit ordering remains the same.

E. Alternatives, applications and tricky viva

Q48. What if a job has deadline zero?

Answer: It has no valid positive-numbered slot before or at its deadline, so the inner loop does not run and the job is skipped.

Q49. What if all jobs have the same deadline?

Answer: Only one job can occupy each slot up to that deadline. The greedy method selects the highest-profit jobs that fit into those available slots.

Q50. What if the maximum deadline is greater than the number of jobs?

Answer: At most n jobs can be scheduled, so slots beyond n are unnecessary. The code remains correct but may allocate more memory than needed; an implementation can cap the useful slot count at min(maxDeadline, n).

Q51. What if every job can be scheduled?

Answer: The total profit is the sum of all job profits, and jobCount equals n. This occurs when the deadlines and available slots permit all jobs to fit.

Q52. Does this algorithm handle jobs with different durations?

Answer: No. The standard greedy proof assumes every job takes one unit of time. Arbitrary processing times require a different scheduling model and algorithm.

Q53. Does it handle multiple machines or processors?

Answer: No. This implementation assumes one machine and one job per time slot. Multiple machines require a different scheduling formulation.

Q54. When might dynamic programming be considered instead?

Answer: When the scheduling problem has additional constraints or a different structure that prevents the standard greedy rule from being optimal. DP is not needed for the classic unit-time, single-machine version.

Q55. What is brute force in this context?

Answer: Brute force could enumerate subsets or possible schedules and select the feasible one with maximum profit. It can find an optimum for small inputs but becomes expensive as the number of jobs grows.

Q56. What are real-world applications or related uses?

Answer: The model is useful for simplified deadline-based task selection, such as choosing one-slot jobs under deadlines with rewards. Real production scheduling, cloud workloads and project planning often include durations, dependencies and resource constraints that need richer algorithms.

Q57. How could this code be made more portable and robust?

Answer: Use vector<Job> instead of a variable-length array, validate n and deadlines, cap slots at min(maxDeadline,n), and use a sufficiently wide integer type if profits may overflow int.

Q58. Why use `int` for totalProfit, and when should it change?

Answer: The sample uses small integer profits, so int is sufficient. If the sum can exceed the int range, use long long and ensure the input profit type is also wide enough.

Q59. What is an optimal schedule?

Answer: It is a feasible schedule whose total profit is as large as possible among all feasible schedules.

Q60. What should you say if asked why this greedy method is suitable?

Answer: Every job takes one unit of time and earns its profit when completed by its deadline. Sorting by profit prioritizes value, and placing each selected job in its latest feasible free slot preserves earlier slots for tighter deadlines. Under these assumptions, this greedy strategy gives maximum total profit.

Formula, algorithm and complexity sheet

Item	Rule / formula
Objective	Maximize the sum of profits of scheduled jobs
Feasible slot	For deadline d, valid slots satisfy $1 \le j \le d$
Greedy placement	Search j = min(D, deadline) down to 1; take first empty slot
Time	O(n log n + nD), D = maximum deadline
Space	O(n + D) for jobs and slots

Pseudocode

1. Read all jobs (ID, deadline, profit).
2. Sort jobs by descending profit.
3. Find maximum deadline D and initialize slots 1..D as empty.
4. For each job, scan from min(D, job deadline) down to 1.
5. If an empty slot is found, place the job there, add its profit and stop scanning for that job.
6. Display jobs by slot order, job count and total profit.

Last-minute recall

- Sort by descending profit, not deadline.
- Place each selected job in the latest empty slot at or before its deadline.
- Latest placement preserves earlier slots for jobs with tighter deadlines.
- Each job is scheduled at most once; break stops the inner search after placement.
- The output is chronological slot order, not profit-sorted order.
- Time: O(n log n + nD); space: O(n + D).
- The classic proof assumes unit-time jobs on one machine.
- Sample: C A E; 3 jobs; total profit 142.
- DSU can accelerate latest-free-slot lookup to near-constant amortized time.

YouTube video links

Sundeep Saradhi Kanthety – Job Sequencing with Deadlines (English)
https://www.youtube.com/watch?v=ZcvLJCjN8eg

THE GATEHUB – Job Sequencing with Deadline using Greedy Method (Hindi/English)
https://www.youtube.com/watch?v=oekAdRwG1B8

Sudhakar Atchala – Job Sequencing with Deadlines (Greedy Method)
https://www.youtube.com/watch?v=RiYZo5_rWWE